

Problem Statement 5 – Smart Traffic Violation Ticket Issuance & Payment Workflow System

1. Hackathon Objective

This hackathon evaluates students on:

1.1 Backend Skills

- *Java OOP design*
- *Spring Boot REST API development*
- *SQL database modeling*
- *Violation lifecycle workflow management*
- *Fine calculation logic*
- *Role-based authorization*
- *Payment transaction tracking*
- *Business rule enforcement*
- *Dispute and appeal handling*

1.2 Frontend Skills (React.js)

- *Component-based UI development*
- *Role-based dashboards*
- *Form handling & validation*
- *REST API integration using Axios/Fetch*
- *State management (useState/useContext)*
- *Data tables with filters and sorting*
- *Status tracking UI components*
- *Responsive UI design*

2. Problem Statement

*Design and develop a **Full Stack Smart Traffic Violation Ticket Issuance & Payment System** to digitally manage traffic violations, penalties, and fine payments.*

The system should allow:

GUVI

- *Traffic officers to issue violation tickets*
- *System to calculate fine based on violation type*
- *Citizens to view issued tickets*
- *Citizens to pay fines online*
- *Officers to track unpaid violations*
- *Admin to monitor all traffic violations*
- *Citizens to raise dispute or appeal*
- *System to track violation lifecycle*
- *Admin to generate violation analytics reports*

The system must enforce structured workflow ensuring transparent violation tracking and fine payment management.

3. Core Domain Entities

3.1 User

- *Id*
- *Name*
- *Email*
- *phoneNumber*
- *Role*
- *createdAt*

Roles:

- *ADMIN*
- *TRAFFIC_OFFICER*
- *CITIZEN*
- *REVIEW_OFFICER*

3.2 Vehicle

- *Id*
- *vehicleNumber*
- *owner (User – CITIZEN)*
- *vehicleType*

GUVI

- *registrationDate*

Vehicle Types:

Car

Bike

Truck

Bus

3.3 ViolationType

- *Id*
- *violationName*
- *Description*
- *baseFineAmount*
- *createdAt*

Examples:

Signal Jumping

Over Speeding

No Helmet

Wrong Parking

No Seatbelt

3.4 ViolationTicket

- *Id*
- *Vehicle*
- *violationType*
- *issuedBy* (User – *TRAFFIC_OFFICER*)
- *violationDate*
- *violationLocation*
- *fineAmount*
- *dueDate*
- *ticketStatus*
- *createdAt*

Ticket Status Lifecycle:

ISSUED → PENDING_PAYMENT → PAID → OVERDUE → DISPUTED → CANCELLED

GUVI

3.5 PaymentTransaction

- *Id*
- *violationTicket*
- *Amount*
- *paymentDate*
- *paymentStatus*
- *referenceId*
- *createdAt*

Payment Status Lifecycle:

INITIATED → SUCCESS → FAILED

3.6 Dispute

- *Id*
- *violationTicket*
- *raisedBy (User – CITIZEN)*
- *disputeReason*
- *disputeStatus*
- *resolvedBy (User – REVIEW_OFFICER)*
- *resolutionRemark*
- *createdAt*
- *resolvedAt*

Dispute Status Lifecycle:

OPEN → UNDER_REVIEW → APPROVED → REJECTED → CLOSED

4. Workflow Lifecycle (Mandatory)

Ticket Lifecycle

ISSUED → PENDING_PAYMENT → PAID → OVERDUE → DISPUTED → CANCELLED

Payment Lifecycle

INITIATED → SUCCESS → FAILED

GUVI

Dispute Lifecycle

OPEN → UNDER_REVIEW → APPROVED → REJECTED → CLOSED

5. MUST HAVE REQUIREMENTS

5.1 User Management

ADMIN can:

- *Create users with role*
- *View all users*
- *Filter users by role*

5.2 Vehicle Management

CITIZEN can:

- *Register vehicle*
- *View registered vehicles*

ADMIN can:

- *View all vehicles*

5.3 Violation Ticket Management

TRAFFIC_OFFICER can:

- *Issue violation ticket*
- *Select violation type*
- *Enter violation location*
- *Assign vehicle*

SYSTEM must:

- *Calculate fine automatically*
- *Generate unique ticket ID*

GUVI

CITIZEN can:

- *View issued tickets*
- *Track ticket status*

ADMIN can:

- *View all violation tickets*

5.4 Fine Payment Management

CITIZEN can:

- *Pay fine online*
- *View payment history*

SYSTEM must:

- *Update payment status automatically*
- *Mark ticket as PAID after successful payment*

ADMIN can:

- *View all payment transactions*

5.5 Dispute Management

CITIZEN can:

- *Raise dispute against ticket*

REVIEW_OFFICER can:

- *Review dispute*
- *Approve dispute (cancel ticket)*
- *Reject dispute*

ADMIN can:

- *View dispute reports*

GUVI

6. Business Rules (Strictly Enforced)

- *Only TRAFFIC_OFFICER can issue violation ticket*
- *Fine amount must be based on violation type*
- *Payment must be completed before due date*
- *Overdue ticket may include penalty*
- *Ticket cannot be modified after payment*
- *Disputed ticket cannot be paid until resolved*
- *Cancelled ticket cannot accept payment*
- *Vehicle must be registered before ticket issuance*

Fine Calculation Rules

Final Fine Amount calculation example:

FinalFine = BaseFine + Penalty (if overdue)

Penalty Example:

10% additional fine after due date

System must automatically update:

- *ticketStatus*
- *paymentStatus*
- *fineAmount*

7. REST API Requirements

- *Proper HTTP methods*
- *DTO usage*
- *Validation annotations*
- *Structured error responses*
- *Meaningful messages*

Sample APIs

User APIs:

POST /api/users

GUVI

GET /api/users

GET /api/users?role=CITIZEN

Vehicle APIs:

POST /api/vehicles

GET /api/vehicles

Violation APIs:

POST /api/violations

GET /api/violations

Ticket APIs:

POST /api/tickets

GET /api/tickets

GET /api/tickets/{id}

Payment APIs:

POST /api/payments

GET /api/payments

Dispute APIs:

POST /api/disputes

PUT /api/disputes/{id}/resolve

8. Frontend Requirements (React.js)

Pages / Components

Authentication:

- *Login page*

Citizen Dashboard:

- *Register vehicle form*
- *View violation tickets*
- *Pay fine interface*
- *Payment history table*

GUVI

- *Raise dispute form*

Traffic Officer Dashboard:

- *Issue violation ticket form*
- *Violation history table*

Review Officer Dashboard:

- *Dispute review panel*
- *Approve or reject dispute interface*

Admin Dashboard:

- *View all violations*
- *Payment analytics*
- *Dispute reports*

UI Expectations

- *Form validation*
- *API integration using Axios*
- *Status badges (PAID, OVERDUE etc.)*
- *Table view with filters*
- *Payment status indicators*
- *Responsive UI layout*

9. Database Requirements

Minimum Tables:

- *Users*
- *Vehicles*
- *Violation_types*
- *Violation_tickets*
- *Payment_transactions*
- *disputes*

Mandatory:

GUVI

- Foreign key (vehicle → owner)
- Foreign key (ticket → vehicle)
- Foreign key (ticket → violation_type)
- Foreign key (payment → ticket)
- Foreign key (dispute → ticket)

Enum mapping required for:

- Role
- ticketStatus
- paymentStatus
- disputeStatus

Query Requirements

At least one JOIN query:

Example:

Fetch violation ticket with vehicle and owner details

At least one aggregate query:

Examples:

Total fines collected per violation type

Total unpaid fines count

Most common violation type

10. GOOD TO HAVE

- Global exception handling
- Pagination and sorting
- Unit test for fine calculation logic
- Swagger documentation
- Payment summary dashboard
- Notification simulation
- Audit log tracking
- Role-based access validation

GUVI

11. Non-Functional Expectations (Implicit Evaluation)

Understanding requirements before coding

- *Logical API and entity design*
- *Proper workflow lifecycle implementation*
- *Handling payment edge cases*
- *Clear explanation of design decisions*
- *Readable and maintainable code*

12. Evaluation Criteria

Candidates will be evaluated on:

- *Java and Spring Boot knowledge*
- *React component design*
- *Database schema design*
- *REST API clarity*
- *Workflow lifecycle implementation*
- *Problem-solving approach*
- *Code quality*
- *System design clarity*
- *Learning adaptability*